Min-Heap Algorithms Reference Manual

COMPLETE 0-INDEXED SPECIFICATION & PARADIGM SEPARATION

1. INDEXING & CORE ACCESSORS (0-BASED ARRAY INDEXING)

In 0-indexed binary heaps, array index 0 represents the root. Navigation for node `i` is defined as:

```
PARENT(i) = ⌊(i - 1) / 2⌋   LEFT(i) = 2i + 1   RIGHT(i) = 2i + 2   HEAP-MINIMUM(A) = A[0]
```

2. BOTTOM-UP MIN-HEAP OPERATIONS (SIFT-DOWN PARADIGM)

Builds the heap in linear time O(N) by sifting smaller elements down from non-leaf nodes toward leaves.

BOTTOM-UP-MIN-HEAPIFY(A, i) — O(log N)

```
1  l = LEFT(i),  r = RIGHT(i)
2  smallest = i
3  if l < A.heap-size and A[l] < A[smallest]
4      smallest = l
5  if r < A.heap-size and A[r] < A[smallest]
6      smallest = r
7  if smallest != i
8      exchange A[i] with A[smallest]
9      BOTTOM-UP-MIN-HEAPIFY(A, smallest)
```

BUILD-MIN-HEAP-BOTTOM-UP(A) — O(N)

```
1  A.heap-size = A.length
2  for i = ⌊A.length / 2⌋ - 1 downto 0
3      BOTTOM-UP-MIN-HEAPIFY(A, i)
```

BOTTOM-UP-EXTRACT-MIN(A) — O(log N)

```
1  if A.heap-size < 1 error "heap underflow"
2  min_val = A[0]
3  A[0] = A[A.heap-size - 1]
4  A.heap-size = A.heap-size - 1
5  BOTTOM-UP-MIN-HEAPIFY(A, 0)
6  return min_val
```

BOTTOM-UP-HEAP-DELETE(A, i) — O(log N)

```
1  if i < 0 or i >= A.heap-size error "invalid
index"
2  last_val = A[A.heap-size - 1]
3  A.heap-size = A.heap-size - 1
4  if i < A.heap-size
5      A[i] = last_val
6      BOTTOM-UP-MIN-HEAPIFY(A, i)
```

3. TOP-DOWN MIN-HEAP OPERATIONS (SIFT-UP PARADIGM)

Builds the heap incrementally in O(N log N) time by inserting elements and floating them up toward the root.

TOP-DOWN-HEAP-DECREASE-KEY(A, i, key) — O(log N)

```
1  if key > A[i] error "key is larger than A[i]"
2  A[i] = key
3  while i > 0 and A[PARENT(i)] > A[i]
4      exchange A[i] with A[PARENT(i)]
5      i = PARENT(i)
```

TOP-DOWN-MIN-HEAP-INSERT(A, key) — O(log N)

```
1  A.heap-size = A.heap-size + 1
2  A[A.heap-size - 1] = ∞
3  TOP-DOWN-HEAP-DECREASE-KEY(A, A.heap-size - 1,
key)
```

BUILD-MIN-HEAP-TOP-DOWN(A) — O(N log N)

```
1  A.heap-size = 1
2  for i = 1 to A.length - 1
3      TOP-DOWN-HEAP-DECREASE-KEY(A, i, A[i])
```

TOP-DOWN-HEAP-DELETE(A, i) — O(log N)

```
1  TOP-DOWN-HEAP-DECREASE-KEY(A, i, -∞)
2  BOTTOM-UP-EXTRACT-MIN(A)
```

4. HEAP SEARCH & SELECTION ALGORITHMS

Procedures for arbitrary element lookup and identifying order statistics within a binary min-heap structure.

HEAP-SEARCH(A, i, key) (Pruned Traversal) `O(N) Worst`

```
1  if i >= A.heap-size or A[i] > key
2      return -1   // Subtree root exceeds key
3  if A[i] == key
4      return i    // Found key at index i
5  left_res = HEAP-SEARCH(A, 2i + 1, key)
6  if left_res != -1
7      return left_res
8  return HEAP-SEARCH(A, 2i + 2, key)
```

KTH-SMALLEST-MIN-HEAP(A, N, K) `O(N + K log N)`

```
1  BUILD-MIN-HEAP-BOTTOM-UP(A)
2  for step = 1 to K - 1
3      BOTTOM-UP-EXTRACT-MIN(A)
4  return A[0]  // Root holds K-th smallest
element
```

5. COMPLETE PARADIGM & OPERATION COMPARISON

Operation	Bottom-Up Approach	Top-Down Approach	Time Complexity	Primary Primitive Used
Heap Construction	`BUILD-MIN-HEAP-BOTTOM-UP`	`BUILD-MIN-HEAP-TOP-DOWN`	`O(N)` vs `O(N log N)`	`MIN-HEAPIFY` vs `DECREASE-KEY`
Element Insertion	N/A (Bulk Construction)	`TOP-DOWN-MIN-HEAP-INSERT`	`O(log N)`	`HEAP-DECREASE-KEY` (Sift-Up)
Find Minimum	`HEAP-MINIMUM(A)`	`HEAP-MINIMUM(A)`	`O(1)`	Direct Access (`A[0]`)
Extract Minimum	`BOTTOM-UP-EXTRACT-MIN`	Uses Sift-Down Heapify	`O(log N)`	`MIN-HEAPIFY(A, 0)`
Decrease Key	Uses Sift-Up Operation	`TOP-DOWN-HEAP-DECREASE-KEY`	`O(log N)`	Parent Comparisons (Sift-Up)
Arbitrary Deletion	`BOTTOM-UP-HEAP-DELETE`	`TOP-DOWN-HEAP-DELETE`	`O(log N)`	Replace & Heapify / Decrease to -∞
Search (Existing)	`HEAP-SEARCH(A, 0, key)`	`HEAP-SEARCH(A, 0, key)`	`O(1)` Best / `O(N)` Worst	Pruned Recursive Search
Search (Non-Existing)	`HEAP-SEARCH(A, 0, key)`	`HEAP-SEARCH(A, 0, key)`	`O(N)`	Pruned Traversal (Visits nodes ≤ key)
K-th Smallest Selection	`KTH-SMALLEST-MIN-HEAP`	Dynamic Min-Heap Extract	`O(N + K log N)`	Linear Build + K-1 Extractions

6. EXPERIMENTAL & THEORETICAL INSIGHTS FOR REPORT ANALYSIS

1. Construction Efficiency: Bottom-up construction requires at most 2N comparisons and N swaps because the work per node is proportional to node height (majority of nodes are near leaves), making it strictly **O(N)**. Top-down construction performs sifting from the bottom up to the root, resulting in **O(N log N)** on descending input.

2. Input Sensitivity: *Ascending input* is optimal for top-down insertion (0 swaps), while *descending input* causes maximum sifting swaps in top-down mode. Search efficiency depends on key magnitude relative to heap subtree values.